

AWS-NATIVE, STANDARDS-FIRST AGENTIC PLATFORM ARCHITECTURE

Reference Architecture for Conversational AI, Agent Memory,
and Multi-Agent Systems on Amazon Bedrock AgentCore + Strands

Interoperable via MCP, A2A, AG-UI, OpenTelemetry, OpenAPI & AsyncAPI

AGENTCORE

STRANDS 1.0

MCP

A2A

AG-UI

OTEL

ROLE LENSES APPLIED:

Enterprise Architecture . AWS AI/ML . Platform Engineering . Security Architecture . Agentic Systems

9

Architecture Layers

7

Open Standards

15

Anti-Patterns Mapped

2

Medium Lock-In Points

- > Part 1: Architecture Principles & Standards Map
- > Part 2: Layered Reference Architecture (AWS Services)
- > Part 3: Memory, Session & Multi-Agent Design
- > Part 4: Security, Governance & Anti-Lock-In Strategy

TABLE OF CONTENTS

Executive Summary	3
Part 1: Architecture Principles & Standards Map	4
1.1 Design Principles	4
1.2 Standards-to-AWS-Service Mapping	5
1.3 The Anti-Lock-In Contract	6
Part 2: Layered Reference Architecture	7
2.1 Architecture Overview Diagram	7
2.2 Layer 1: Client & Experience (AG-UI)	8
2.3 Layer 2: API & Gateway (OpenAPI/AsyncAPI)	8
2.4 Layer 3: Agent Runtime (AgentCore + Strands)	9
2.5 Layer 4: Tool & Integration (MCP)	10
2.6 Layer 5: Multi-Agent Coordination (A2A)	11
2.7 Layer 6: Memory & Knowledge	12
2.8 Layer 7: Data & Storage	13
2.9 Layer 8: Observability (OpenTelemetry)	14
2.10 Layer 9: Security & Governance	15
Part 3: Memory, Session & Multi-Agent Design	16
3.1 Conversation & Session Persistence on AWS	16
3.2 Memory Architecture with AgentCore Memory	17
3.3 Multi-Agent Topology with Strands + A2A	18
3.4 Agent Recovery & Durable Execution	19
Part 4: Security, Governance & Anti-Lock-In Strategy	20
4.1 Identity, Auth & Tenant Isolation	20
4.2 Governance & Compliance Mapping	20
4.3 Anti-Pattern Cross-Reference	21
4.4 Portability & Exit-Strategy Design	22
Appendix: Service & Standards Reference Tables	23
Appendix: Infrastructure-as-Code Skeleton	25

Executive Summary

This document specifies a reference architecture for a production-grade conversational AI and multi-agent platform that is AWS-native by default but standards-first by design. Amazon Bedrock AgentCore provides the managed runtime, memory, gateway, and observability substrate; Strands Agents (1.0) provides the agent orchestration framework. Every external boundary of the system—client communication, tool access, agent-to-agent coordination, API contracts, and telemetry—is implemented against an open standard (AG-UI, MCP, A2A, OpenAPI, AsyncAPI, OpenTelemetry) so that AWS-specific services can be replaced without rewriting the agents, tools, or client applications that depend on them.

Architectural Thesis

The thesis is straightforward: AWS services should sit *behind* standard protocol boundaries, never define them. AgentCore Runtime hosts agents, but agents are invoked via A2A-compliant task requests. AgentCore Gateway exposes tools, but those tools are described via MCP. AgentCore Memory persists state, but the memory access pattern is abstracted so an alternative memory provider (mem0, Letta, a self-hosted store) could be substituted. This is not a hedge against AWS—it is the pattern that makes the platform composable, auditable, and resilient to both vendor and protocol churn.

Key Architectural Decisions

Concern	Primary Choice	Rationale
Runtime	Amazon Bedrock AgentCore Runtime (primary)	Managed, serverless, checkpointed agent execution with session isolation; framework-agnostic so Strands is a choice, not a constraint.
Agent Framework	Strands Agents 1.0 (Python/TypeScript)	AWS-built, open-source (Apache 2.0), model-agnostic, native A2A + OTel support; multi-agent orchestration as of 1.0 (May 2026).
Tool Access	MCP via AgentCore Gateway	Every tool—AWS-native or third-party—exposed as an MCP server; agents and IDEs consume identical tool definitions.
Agent Coordination	A2A Protocol	Cross-agent and cross-vendor task delegation; AgentCore Runtime agents are A2A-addressable endpoints.
Client Interface	AG-UI Protocol	Event-driven, bidirectional agent-to-frontend streaming; decouples UI framework from agent runtime.
API Contracts	OpenAPI 3.1 + AsyncAPI 3.0	Synchronous tool/service APIs documented in OpenAPI; event-driven and streaming interfaces in AsyncAPI.
Observability	OpenTelemetry -> AWS Distro for OTel (ADOT)	Vendor-neutral instrumentation; AWS collector as default export target, swappable for Honeycomb/Datadog/Grafana.
Memory	AgentCore Memory + abstraction layer	Short-term (session) and long-term (semantic/episodic) memory via managed service, accessed through a provider-agnostic interface.

INFO: Reading guide: Part 1 establishes principles and the standards-to-service mapping that everything else derives from. Part 2 walks the architecture layer by layer with concrete AWS service choices. Part 3 details memory, session persistence, and multi-agent design—the hardest problems identified in prior research. Part 4 covers security, governance, and the specific mechanisms that prevent lock-in from becoming a liability.

PART 1

Architecture Principles & Standards Map

Eight principles govern every decision in this architecture. They are written as testable constraints, not aspirations.

1.1 Design Principles

P1 — Protocols at the boundary, services behind it

Every external interface (client, tool, agent-to-agent, telemetry) is defined by an open standard. AWS services implement one side of that interface; they never become the interface itself. Test: could a non-AWS implementation of this component be swapped in by changing configuration, not code, at the consuming layer?

P2 — AWS-native is the default, not the constraint

Where AWS offers a managed service that implements a standard well (AgentCore Runtime for agent hosting, AgentCore Gateway for MCP), use it—operational simplicity matters. But the choice is justified by standards compliance plus operational fit, not by AWS being the only option considered.

P3 — Memory and conversation state are portable by construction

Conversation, session, and long-term memory records are stored in formats and schemas that do not depend on a single vendor's proprietary memory API. AgentCore Memory is the primary store, accessed through an internal abstraction that could be re-pointed at another provider (Section 3.2).

P4 — Every agent is independently addressable and replaceable

Agents are A2A endpoints first, AgentCore Runtime workloads second. A specialized agent built on a different framework (LangGraph, CrewAI, Claude Agent SDK) can participate in the same multi-agent system as long as it exposes an A2A-compliant interface.

P5 — Observability is structural, not bolted on

OpenTelemetry instrumentation is present from the first agent, exported through ADOT to a vendor-neutral pipeline. Trace, span, and log correlation IDs flow through A2A and MCP calls so a single request can be reconstructed end-to-end regardless of which AWS service or external agent handled which step.

P6 — Governance is encoded in infrastructure, not policy documents

Tenant isolation, data residency, deletion cascades, and audit logging are implemented as infrastructure constraints (IAM policies, resource tagging, automated pipelines)—not as procedures that depend on engineers remembering to follow them.

P7 — Durability is assumed for any multi-step agent task

Any agent workflow involving more than one tool call or external side effect runs under AgentCore Runtime's checkpointing, with idempotency keys on state-changing tool calls. 'It might fail partway through' is a default assumption, not an edge case.

P8 — Anti-patterns from prior research are design constraints

The fifteen anti-patterns catalogued in prior research (context window dependence, unlimited memory growth, missing checkpointing, cross-tenant query leakage, etc.) map directly to specific mitigations in this architecture (Section 4.3). Each is a named constraint with an owner, not a general caution.

1.2 Standards-to-AWS-Service Mapping

This is the master cross-reference: for each open standard, which AWS service implements the AWS-native path, what the standard guarantees regardless of implementation, and what the swap-out path looks like if the AWS service is later replaced.

Standard	AWS-Native Implementation	What the Standard Guarantees	Swap-Out Path
MCP (Model Context Protocol)	AgentCore Gateway exposes Lambda functions, APIs, and AWS services as MCP tool servers	Tool definitions, schemas, and invocation semantics are identical for any MCP client (Claude, IDE agents, Strands agents)	Point agents at any MCP-compliant gateway (self-hosted, Anthropic's reference implementation, third-party); tool contracts unchanged
A2A (Agent2Agent Protocol)	AgentCore Runtime agents (via Strands 1.0) expose A2A task endpoints natively	Any A2A-compliant agent—regardless of framework or host—can be discovered and delegated to	Replace AgentCore-hosted agents with agents on other runtimes (self-hosted, other clouds); A2A task contracts unchanged
AG-UI (Agent-UI Protocol)	Client applications connect to AgentCore Runtime via an AG-UI-compliant gateway layer (API Gateway + Lambda adapter)	Frontend frameworks (React, custom clients) consume a standard event stream regardless of backend agent implementation	Swap the agent backend (different AgentCore agent, different framework entirely) without frontend changes
OpenTelemetry	ADOT (AWS Distro for OpenTelemetry) Collector, with Strands' native OTel instrumentation	Traces, metrics, and logs use OTel semantic conventions; correlation IDs are vendor-neutral	Re-point the ADOT collector's exporter from CloudWatch/X-Ray to Honeycomb, Datadog, Grafana Cloud, or self-hosted Jaeger
OpenAPI 3.1	API Gateway REST/HTTP APIs defined via OpenAPI specs; AgentCore Gateway tool definitions derived from OpenAPI where applicable	Synchronous service contracts are machine-readable and codegen-compatible across any API gateway	Import the same OpenAPI spec into another API gateway (Kong, Apigee, self-hosted)
AsyncAPI 3.0	EventBridge, SQS, and AppSync Events (for streaming) document event contracts via AsyncAPI	Event-driven and streaming contracts (agent status updates, async task completion) are documented independent of the broker	Re-implement the event contract on Kafka, NATS, or another broker using the same AsyncAPI spec
MCP Memory Extensions / Provider-agnostic memory API	AgentCore Memory (short-term session + long-term semantic/episodic stores)	Internal memory abstraction layer presents a consistent read/write/search interface to agents regardless of backing store	Re-point the abstraction layer at mem0, Letta, or a custom Postgres+pgvector store; agent code unchanged

1.3 The Anti-Lock-In Contract

Lock-in is not binary—it is a spectrum of switching cost. This architecture does not pretend switching away from AWS is free; it ensures the switching cost is proportional to genuine value received (operational simplicity, managed scaling, integrated security) rather than artificial cost imposed by proprietary interfaces. The contract below states, for each major AWS service used, what would need to change to replace it.

AWS Service	Lock-In Level	Replacement Path	What Changes
Amazon Bedrock AgentCore Runtime	Medium	Re-host agent containers on another A2A-compliant runtime (self-managed ECS/EKS + open-source A2A server, or another cloud's agent runtime). Strands agent code is unchanged—Strands is framework-agnostic and runs anywhere.	Checkpointing/recovery semantics must be re-implemented (e.g., via Temporal or LangGraph persistence) since AgentCore's managed checkpointing is proprietary.
Amazon Bedrock AgentCore Gateway	Low	Any MCP-compliant gateway can replace it. Tool definitions (MCP schemas) are portable by design.	Re-point MCP client configuration; no agent code changes.
Amazon Bedrock AgentCore Memory	Medium	Re-point the internal memory abstraction layer at mem0, Letta, or self-hosted Postgres+pgvector/OpenSearch.	Bulk export/migration of existing memory records required; abstraction layer interface unchanged so agent code is unaffected.
Amazon Bedrock (model access)	Low	Strands is model-agnostic; swap the model provider configuration (Anthropic direct, OpenAI, self-hosted via SageMaker/Bedrock Marketplace).	Prompt tuning may be needed across model families; no architectural change.
Amazon API Gateway / AppSync	Low	Any API gateway that can host OpenAPI/AsyncAPI-defined routes and an AG-UI adapter.	Re-deploy gateway configuration from IaC; client-facing contracts (AG-UI, OpenAPI) unchanged.
Amazon CloudWatch / X-Ray (via ADOT)	Very Low	Re-point ADOT Collector exporter configuration to any OTLP-compatible backend.	No code change; configuration-only switch.
Amazon DynamoDB / Aurora (conversation store)	Medium	Schema is standard relational/document design (per prior research data models); export and re-import to PostgreSQL, CockroachDB, or another store.	Migration effort proportional to data volume; no architectural redesign needed since schema was designed store-agnostic.

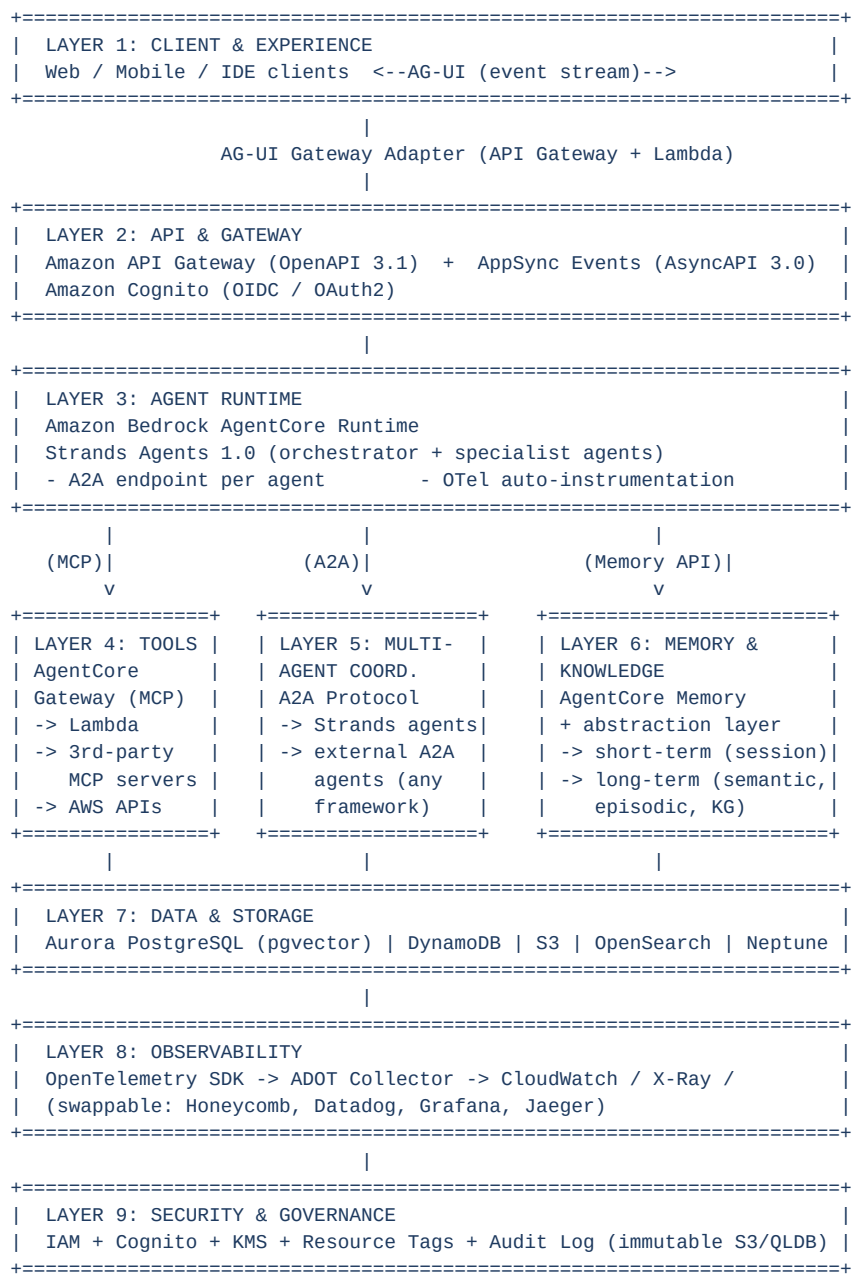
DECISION: Net assessment: the highest lock-in points (AgentCore Runtime's managed checkpointing, AgentCore Memory's storage format) are also the components delivering the most operational value—this is an acceptable, deliberate trade rather than an oversight. The architecture ensures these are the **only** two components with non-trivial switching cost, isolated behind the abstraction layers described in Sections 2.4 and 3.2.

PART 2

Layered Reference Architecture

Nine layers, each with a standards-defined interface and a primary AWS implementation. Layers communicate only through their declared standard—no layer reaches into another layer's internals.

2.1 Architecture Overview



Layers 4, 5, and 6 are peers, not a stack—the agent runtime calls each independently via its respective protocol. This is the key structural decision: tools (MCP), other agents (A2A), and memory are three distinct integration surfaces, each governed by a different standard, each independently swappable.

2.2 Layer 1: Client & Experience (AG-UI)

AG-UI (CopilotKit, open protocol since early 2025) standardizes bidirectional, event-driven communication between agents and frontends—agent state updates, streaming tokens, tool-call visibility, and human-in-the-loop interrupts all flow over a single typed event stream.

- **AWS implementation:** An AG-UI adapter (Lambda behind API Gateway WebSocket or HTTP/2 streaming) translates between AgentCore Runtime's native streaming response format and AG-UI event types (RUN_STARTED, TEXT_MESSAGE_CONTENT, TOOL_CALL_START, STATE_DELTA, etc.).
- **Why not expose AgentCore's native protocol directly:** doing so would couple every client (web, mobile, IDE plugin, third-party integration) to AWS-specific message formats. The AG-UI adapter is a thin, stateless translation layer—cheap to maintain, and it means any AG-UI-compatible frontend component library works unmodified.
- **Human-in-the-loop:** AG-UI's interrupt/resume event types map directly onto AgentCore Runtime's checkpoint/resume primitives (Section 3.4)—an approval request pauses the agent, the checkpoint persists, and resumption is a normal AG-UI resume event.

2.3 Layer 2: API & Gateway (OpenAPI / AsyncAPI)

All synchronous request/response contracts (auth, conversation CRUD, artifact retrieval) are defined in OpenAPI 3.1. All asynchronous contracts (agent run status events, long-task completion notifications, streaming progress) are defined in AsyncAPI 3.0. Both specs are checked into source control and drive code generation for client SDKs.

- **Amazon API Gateway (HTTP API)** hosts the OpenAPI-defined synchronous routes; the spec is uploaded as the API Gateway definition source, keeping the gateway configuration and the contract in lock-step.
- **Amazon Cognito** issues OAuth2/OIDC tokens validated by API Gateway authorizers—standard JWT validation, portable to any OIDC-compliant identity provider (Auth0, Okta, Keycloak) by reconfiguring the authorizer, not rewriting the API.
- **AWS AppSync Events** (or EventBridge + WebSocket API) implements the AsyncAPI-defined event channels for agent status streaming when AG-UI's own transport isn't used directly by a consumer (e.g., backend-to-backend integrations).
- **Why this matters:** a partner integrating via API never needs to know AgentCore exists. They consume OpenAPI/AsyncAPI specs—identical to specs a non-AWS implementation would publish.

2.4 Layer 3: Agent Runtime (AgentCore + Strands)

This is the heart of the platform. Amazon Bedrock AgentCore Runtime provides the managed, serverless execution environment—session isolation, checkpointing, scaling to thousands of concurrent invocations. Strands Agents 1.0 (Apache 2.0, AWS open-source) provides the agent loop, multi-agent orchestration primitives, and native OpenTelemetry + A2A support.

Why AgentCore Runtime

- Framework-agnostic by design—AgentCore documentation explicitly supports Strands, LangGraph, CrewAI, and custom frameworks packaged as containers; choosing Strands is a recommendation, not a requirement imposed by the runtime.
- Managed checkpointing and recovery: agents resume from interruption without custom durable-execution code (directly addresses anti-pattern AP-07, Missing Checkpointing, from prior research).
- Session isolation per invocation: each conversation/session runs in an isolated execution context, addressing cross-tenant leakage risk (AP-15) at the runtime level.

Why Strands Agents 1.0

- Model-agnostic: works with Anthropic, OpenAI, Meta Llama, and self-hosted models via Bedrock or direct API—no architectural dependency on a single model provider.
- Native multi-agent orchestration (as of v1.0, May 2026) including A2A protocol support and a remote session manager—directly enables Layer 5 (Section 2.6) without bespoke glue code.
- Native OpenTelemetry instrumentation—Layer 8 observability (Section 2.9) is populated automatically, not bolted on.
- Open source (Apache 2.0) with broad industry contribution (Accenture, Anthropic, Meta, PwC, Langfuse, mem0.ai, Tavily)—reduces single-vendor risk in the framework layer itself, distinct from the runtime layer.

AWS NATIVE: Agent packaging pattern: each Strands agent (orchestrator and specialists) is packaged as a container image with a standard entrypoint. AgentCore Runtime hosts the container; the container itself has zero AgentCore-specific imports beyond a thin runtime adapter shim. This shim is the only file that would need to change to run the same container on, e.g., self-managed Fargate with an open-source A2A server.

Agent container structure (illustrative)

```
agent-orchestrator/
+-- Dockerfile
+-- pyproject.toml           # strands-agents, opentelemetry-* deps
+-- src/
|   +-- main.py              # entrypoint; AgentCore runtime adapter
|   +-- agent.py             # Strands Agent definition (model, prompt, tools)
|   +-- tools/
|       | +-- mcp_client.py   # MCP client config (Layer 4)
|       +-- a2a/
|           | +-- delegate.py  # A2A client for sub-agent calls (Layer 5)
|           +-- memory/
|               | +-- memory_adapter.py  # provider-agnostic memory interface (Layer 6)
|               +-- observability/
|                   +-- otel_setup.py     # OTel SDK init, ADOT exporter config
+-- runtime_adapter/
    +-- agentcore_shim.py      # <- ONLY AgentCore-specific file
```

2.5 Layer 4: Tool & Integration (MCP)

Every capability an agent can invoke—AWS service calls, internal microservices, third-party SaaS integrations—is exposed as an MCP tool server. Amazon Bedrock AgentCore Gateway is the primary mechanism for converting existing AWS resources (Lambda functions, OpenAPI-described APIs) into MCP-compliant tool servers without rewriting them.

- **AgentCore Gateway** ingests an OpenAPI spec or Lambda function ARN and produces an MCP server exposing each operation as a tool—this is the bridge between Layer 2's OpenAPI contracts and Layer 3's tool-calling agents.
- **Third-party MCP servers** (e.g., a CRM, a ticketing system, a code-search tool) are connected directly—AgentCore Gateway can proxy/aggregate multiple MCP servers behind one endpoint, but agents can also connect to external MCP servers directly via Strands' MCP client support.
- **Tool governance:** AgentCore Gateway's IAM integration means tool access is scoped per-agent (or per-session) using standard AWS IAM policies—an agent's MCP tool list is the intersection of 'tools the gateway knows about' and 'tools this agent's IAM role permits'.
- **Idempotency (addresses AP-13):** state-changing MCP tools (create/update/delete operations) are designed with idempotency-key parameters from the OpenAPI spec stage—Lambda implementations check a DynamoDB idempotency table before executing side effects.

MCP tool manifest example

```
{
  "name": "create_support_ticket",
  "description": "Create a ticket in the support system",
  "inputSchema": {
    "type": "object",
    "properties": {
      "title": {"type": "string"},
      "description": {"type": "string"},
      "priority": {"type": "string", "enum": ["low", "medium", "high"]},
      "idempotency_key": {
        "type": "string",
        "description": "Client-generated UUID; required for safe retries"
      }
    }
  },
  "required": ["title", "description", "idempotency_key"]
}
// Backed by: AgentCore Gateway -> Lambda -> DynamoDB idempotency check
//           -> Ticketing system API (OpenAPI-described)
```

2.6 Layer 5: Multi-Agent Coordination (A2A)

Strands 1.0's native A2A support means every agent—orchestrator or specialist—is an A2A-addressable endpoint by default. Multi-agent coordination (manager/planner/research/coding/reviewer patterns from prior research) is implemented as A2A task delegation between Strands agents, each independently hosted on AgentCore Runtime.

- **Discovery:** agents publish A2A 'agent cards' (capability descriptions) to an internal registry (can be as simple as a DynamoDB table or as rich as a dedicated A2A registry service); the orchestrator discovers specialist agents by capability, not by hardcoded endpoint.
- **Cross-framework interoperability:** because A2A is the coordination contract, a specialist agent built on LangGraph, CrewAI, or the Claude Agent SDK can be delegated to identically—as long as it exposes an A2A task endpoint. This directly satisfies the requirement to avoid framework lock-in at the multi-agent layer.
- **Cross-organization patterns:** A2A's HTTP-based transport and task lifecycle (synchronous, streaming, or async-with-notification) support delegating to agents outside the platform entirely—a partner's agent, a different business unit's agent mesh—using the same protocol as internal delegation.
- **Context compaction (addresses prior research finding A.6):** sub-agents return compact A2A task results (structured summaries) to the orchestrator rather than raw tool output, keeping the orchestrator's context window focused on decision-making.

A2A task delegation flow (Strands orchestrator -> specialist)

1. Orchestrator agent (Strands, AgentCore Runtime) receives user request
2. Orchestrator's planning step identifies need for "financial analysis"
3. Orchestrator queries agent registry for capability="financial-analysis"
-> resolves to Specialist Agent X (A2A endpoint, also on AgentCore Runtime,
OR a different team's agent on a different runtime entirely)
4. Orchestrator sends A2A task request:
POST https://specialist-x.internal/a2a/tasks
{ "input": {...}, "context_id": "<conversation_id>",
"callback": "<orchestrator A2A endpoint>" }
5. Specialist Agent X executes (own AgentCore session, own memory,
own OTel trace span linked via traceparent header)
6. Specialist returns A2A task result (structured summary, not raw data)
7. Orchestrator incorporates result into its own context (compacted)
8. OTel trace shows: orchestrator span -> A2A call span -> specialist span
-> specialist's internal tool-call spans (full trace, Section 2.9)

2.7 Layer 6: Memory & Knowledge

Amazon Bedrock AgentCore Memory provides both short-term (session/working) memory and long-term (semantic, episodic) memory as managed services. Per Principle P3, agents access memory exclusively through an internal abstraction layer—the memory taxonomy from prior research (working, episodic, semantic, procedural, project, organizational, agent) maps onto AgentCore Memory's primitives plus supplementary stores where needed.

Memory Type	Primary Store	Access Pattern	Notes
Working / Session	AgentCore Memory (short-term)	Read/write via memory adapter, scoped to session ID	Auto-managed lifecycle tied to AgentCore Runtime session
Episodic	AgentCore Memory (long-term) + Aurora (pgvector)	Semantic search via adapter; embeddings in pgvector for hybrid search	Provenance tag (Section 3.2) stored alongside each record
Semantic (user facts)	AgentCore Memory (long-term)	Extracted async post-conversation; conflict-checked on write	TTL + importance score fields per prior-research best practice D.1
Procedural	S3 (versioned) + Aurora metadata	Agent playbooks/prompts retrieved by orchestrator at session start	Versioned like artifacts; not part of AgentCore Memory
Project / Org	Aurora + OpenSearch (full-text) + S3 (documents)	Project-scoped retrieval; shared across conversations in a project	Maps to 'Projects' construct from prior research Part 10
Knowledge Graph	Amazon Neptune (optional, Graph RAG)	Entity/relationship traversal combined with vector search	Added when multi-hop retrieval quality (Part A.3, prior research) justifies the investment

Memory abstraction interface (illustrative)

```
class MemoryProvider(Protocol):
    # Provider-agnostic interface. AgentCore Memory is the default
    # implementation; mem0/Letta/custom Postgres implementations satisfy
    # the same interface for swap-out (Section 1.3).

    async def write(self, record: MemoryRecord) -> str:
        # record includes: content, type (episodic|semantic|procedural),
        # provenance (user_stated|agent_inferred|tool_derived),
        # importance_score, valid_from, valid_until (optional)
        ...

    async def search(self, query: str, scope: MemoryScope,
                    top_k: int = 5) -> list[MemoryRecord]:
        # scope = session | project | user | org
        ...

    async def check_conflicts(self, record: MemoryRecord) -> list[MemoryRecord]:
        # Retrieve potentially-contradictory existing records
        # before write (mitigates AP-11, No Conflict Resolution)
        ...

# Default binding:
provider: MemoryProvider = AgentCoreMemoryAdapter(memory_id=...)

# Swap-out binding (no agent code changes required):
# provider: MemoryProvider = Mem0Adapter(config=...)
# provider: MemoryProvider = PostgresPgvectorAdapter(dsn=...)
```

DECISION: This abstraction is the single most important anti-lock-in mechanism for memory (Section 1.3). It costs one interface definition and a thin adapter; it buys the ability to migrate memory infrastructure—driven by cost, capability, or compliance—without touching agent logic.

2.8 Layer 7: Data & Storage

Storage selections follow the storage taxonomy established in prior research (Part 1, Section 1.1): relational for metadata/transactions, document/object for content, vector for semantic search, graph for relationships, event log for audit/CQRS.

Storage Need	AWS Service	Standard/Format	Portability Note
Conversation & message metadata	Aurora PostgreSQL (Serverless v2)	Standard SQL schema (per prior research Part 1.2)	pg_dump/restore to any Postgres-compatible target
Vector embeddings	Aurora PostgreSQL + pgvector extension	pgvector (open-source extension)	Standard pgvector; portable to any Postgres with the extension
Large artifacts (code, docs, images)	Amazon S3	Content-addressable, versioned objects	S3 API is a de facto standard; S3-compatible APIs (MinIO, R2) available everywhere
Full-text & hybrid search	Amazon OpenSearch Service	OpenSearch (Apache 2.0, Elasticsearch-API-compatible fork)	OpenSearch is itself open-source; self-hostable
Knowledge graph (optional)	Amazon Neptune	Gremlin / openCypher (open query languages)	Query language portable to Neo4j, JanusGraph, etc.
Event log / audit trail	Amazon EventBridge -> S3 (WORM)	CloudEvents-formatted events (CNCF standard)	CloudEvents format is broker-agnostic
Session cache (hot path)	Amazon ElastiCache (Redis OSS)	Redis protocol (open-source)	Standard Redis; portable to any Redis-compatible cache

PRINCIPLE: Every storage choice in this layer uses either an open-source engine (Postgres, OpenSearch, Redis) run as a managed AWS service, or an open data format (CloudEvents, Gremlin/openCypher) on a proprietary engine (Neptune). This is the practical version of 'standards-first': pay AWS for operational convenience, but never for a proprietary data format you cannot extract.

2.9 Layer 8: Observability (OpenTelemetry)

Every component—AG-UI adapter, API Gateway/Lambda, Strands agents, MCP tool calls, A2A delegations, memory adapter calls—emits OpenTelemetry traces, metrics, and logs using OTel semantic conventions for generative AI (`gen_ai.*` attributes). The AWS Distro for OpenTelemetry (ADOT) Collector is the default export path; this is purely an export destination choice, not an instrumentation choice.

- **Trace correlation across protocol boundaries:** traceparent (W3C Trace Context) headers propagate through AG-UI events, A2A task requests, and MCP tool calls—a single user request produces one trace spanning the orchestrator, every delegated specialist agent, and every tool invocation, regardless of which AWS service or external system handled each span.
- **Strands' native instrumentation** emits spans for the agent loop (model invocation, tool selection, tool execution) automatically—this directly populates the 'tool trace persistence' data model from prior research Part 6 without custom instrumentation code.
- **Sampling policy (addresses AP-06, Trace Explosion):** ADOT Collector applies tail-based sampling—100% retention for traces containing errors or exceeding a latency threshold, 1-5% sampling for fully successful traces—configured once at the collector, not per-service.
- **Export targets:** ADOT Collector ships traces to AWS X-Ray and CloudWatch by default; metrics to CloudWatch; logs to CloudWatch Logs. Re-pointing to Honeycomb, Datadog, Grafana Cloud, or self-hosted Jaeger/Prometheus/Loki is an ADOT Collector configuration change (OTLP exporter endpoint + credentials)—zero application code changes.

OTel resource & span attributes (`gen_ai` semantic conventions)

```
# Span: agent invocation
gen_ai.system = "strands"
gen_ai.agent.name = "orchestrator"
gen_ai.request.model = "anthropic.claude-sonnet-4-6"
gen_ai.operation.name = "chat"
session.id = "<conversation_id>"
enduser.id = "<user_id>"           # hashed/pseudonymized per Section 4.1
tenant.id = "<workspace_id>"       # critical for AP-15 mitigation

# Child span: MCP tool call
gen_ai.tool.name = "create_support_ticket"
gen_ai.tool.call.id = "<idempotency_key>"

# Child span: A2A delegation
peer.service = "specialist-financial-analysis"
a2a.task.id = "<task_id>"
traceparent = "00-<trace_id>-<span_id>-01"  # propagated to specialist
```

2.10 Layer 9: Security & Governance

Security is enforced at every layer boundary using AWS IAM as the policy engine, but the policies themselves encode standards-defined identities (OIDC subjects, A2A agent identities, MCP tool scopes)—so the *rules* are portable even though the *enforcement engine* is AWS-specific.

- **Identity:** Amazon Cognito issues OIDC tokens; AgentCore Runtime sessions, A2A task requests, and MCP tool calls all carry the originating user's OIDC subject claim through the entire call chain—every span in Layer 8 can be attributed to a user and tenant.
- **Tenant isolation:** every Aurora query, OpenSearch query, and AgentCore Memory call is scoped by `tenant_id` at the query level (not post-filtered)—directly mitigating AP-15 (Cross-Tenant Query Without Scoping). IAM policies additionally enforce resource-level tenant boundaries for S3 and DynamoDB.
- **Encryption:** AWS KMS customer-managed keys (CMKs), one per tenant for regulated workloads, encrypt Aurora, S3, and AgentCore Memory at rest. CMK rotation and per-tenant key isolation support both GDPR/CCPA requirements and customer-managed-key compliance demands.
- **Audit trail:** every state-changing action (memory write, tool call with side effects, agent run completion) emits a CloudEvents-formatted audit event to EventBridge, archived to an S3 bucket with Object Lock (WORM) for immutability—satisfying the 'immutable audit log' requirement from prior research Part 13.
- **Deletion cascade (addresses AP-14):** a single 'delete tenant/user data' Step Functions workflow orchestrates deletion across Aurora, S3, OpenSearch, AgentCore Memory, and Neptune—triggered by a DeletionRequest record (per prior research data model) and tracked to completion with SLA monitoring.

AWS NATIVE: Guardrails: Amazon Bedrock Guardrails (content filtering, PII redaction, topic restrictions) are applied at the model-invocation layer within Strands agents. Guardrail policies are configuration, not code—but the *outcomes* (what gets filtered, what gets redacted) should be specified in a portable policy format (e.g., a YAML policy document) so the same governance intent could be re-implemented against another provider's moderation layer if needed.

PART 3

Memory, Session & Multi-Agent Design

This section translates the conversation persistence, memory taxonomy, and multi-agent patterns from prior research into concrete AgentCore + Strands implementations, with explicit treatment of the open research problems identified previously (provenance, temporal validity, fine-grained recovery).

3.1 Conversation & Session Persistence on AWS

AgentCore Runtime provides session-scoped execution state; this is necessary but not sufficient for the full conversation persistence model (sidebar, search, branching, projects) from prior research. The platform splits responsibilities: AgentCore Runtime owns *active execution state* for the duration of a session; Aurora PostgreSQL owns *durable conversation history* that outlives any single AgentCore session.

Concern	Owner	Mechanism
Active agent execution state (mid-turn)	AgentCore Runtime session	Managed checkpointing; ephemeral, tied to active invocation
Durable message history	Aurora PostgreSQL (conversations/messages tables)	Written by orchestrator agent after each turn via a 'persist turn' tool call (MCP)
Conversation sidebar, search, pinning, folders	Aurora PostgreSQL + OpenSearch	Standard CRUD API (Layer 2, OpenAPI); independent of agent runtime
Rolling summaries (context compression)	Generated by orchestrator agent, stored in Aurora	Async Lambda triggered when conversation exceeds token threshold, per prior research Section 2.2
Session resume after long idle period	Context assembly pipeline (Lambda)	Reconstructs layered context (recent verbatim + summary + retrieved memory) per prior research Section 2.1, passed to a fresh AgentCore session

On session resume, the platform does not attempt to 'reopen' the original AgentCore session (sessions are not designed for indefinite persistence). Instead, the context assembly pipeline builds the layered context document exactly as specified in prior research (Part 2, Section 2.1)—recent messages verbatim, rolling summary, retrieved memories—and starts a new AgentCore session seeded with that context. This keeps AgentCore's session model simple (bounded-duration execution) while preserving the user-facing experience of a continuous conversation.

DECISION: This split is itself an anti-lock-in decision: if AgentCore Runtime's session model changes, or the platform migrates to another A2A-compliant runtime, the durable conversation store (Aurora) and context assembly pipeline (a portable Lambda) are unaffected. Only the 'start a new session with this context' call changes.

3.2 Memory Architecture with AgentCore Memory

Building on the memory taxonomy and the abstraction interface introduced in Section 2.7, this section addresses the two open research problems most relevant to memory: provenance (B.1) and temporal validity (B.2).

Provenance tracking (addresses Open Problem B.1)

Every record written through the MemoryProvider interface carries a mandatory provenance field. This is enforced at the abstraction layer—not optional, not added later—so the schema-level cost identified in prior research ('the field is cheap now and expensive to add later') is paid upfront.

```
-- Memory record (stored via AgentCore Memory long-term store,
-- mirrored to Aurora for queryability/audit)
{
  "memory_id": "uuid",
  "tenant_id": "uuid",
  "user_id": "uuid",
  "type": "episodic | semantic | procedural",
  "content": "string",
  "embedding": "vector(1536)",

  -- PROVENANCE (mandatory, enforced at write-time)
  "provenance": {
    "source": "user_stated | agent_inferred | tool_derived",
    "source_detail": "e.g. tool name, agent id, or 'explicit user message'",
    "trust_tier": 1 | 2 | 3,    -- 1=user-stated (highest), 3=tool-derived
    "review_status": "unreviewed | auto_approved | human_reviewed"
  },

  -- TEMPORAL VALIDITY (addresses Open Problem B.2)
  "valid_from": "timestamp",
  "valid_until": "timestamp | null",
  "superseded_by": "memory_id | null",

  "importance_score": 0.0-1.0,
  "created_at": "timestamp",
  "ttl": "timestamp | null"
}
```

Conflict detection & temporal validity (addresses Open Problem B.2)

On every semantic-memory write, the memory adapter calls `check_conflicts()` (Section 2.7) before insertion. If a semantically similar existing record is found:

1. If the new record's provenance `trust_tier` is equal or higher (e.g., a new user-stated fact vs. an old agent-inferred one), the existing record's `valid_until` is set to the new record's `valid_from`, and `superseded_by` is set to the new record's ID. The old record is retained (not deleted) for audit purposes but excluded from default retrieval.
2. If the new record's `trust_tier` is lower (e.g., a tool-derived inference contradicting a user-stated fact), the new record is written with `review_status='unreviewed'` and surfaced for human or async-LLM review rather than silently accepted—directly preventing memory poisoning (AP-04).
3. If no conflict is detected, the record is written normally with `review_status='auto_approved'` for tier-1/2 sources, or 'unreviewed' for tier-3 (tool-derived) sources pending a periodic review batch job.

DECISION: This does not fully solve Open Problem B.2 (no system reviewed achieves complete temporal-validity modeling)—but it converts an unbounded research problem into a bounded engineering one: conflict detection runs on every write (doubling retrieval cost on write, as prior research noted), and the trust-tier ordering gives a deterministic resolution rule for the common cases, with human review as the fallback for ambiguous ones.

Memory scope & namespace isolation

AgentCore Memory namespaces map to the scope hierarchy from prior research (Part 7.2 / Part 10): session -> project -> user -> organization. Namespace IDs are constructed to include tenant_id as a prefix, so even a misconfigured query is structurally incapable of crossing tenant boundaries (defense-in-depth alongside the query-level scoping from Section 2.10).

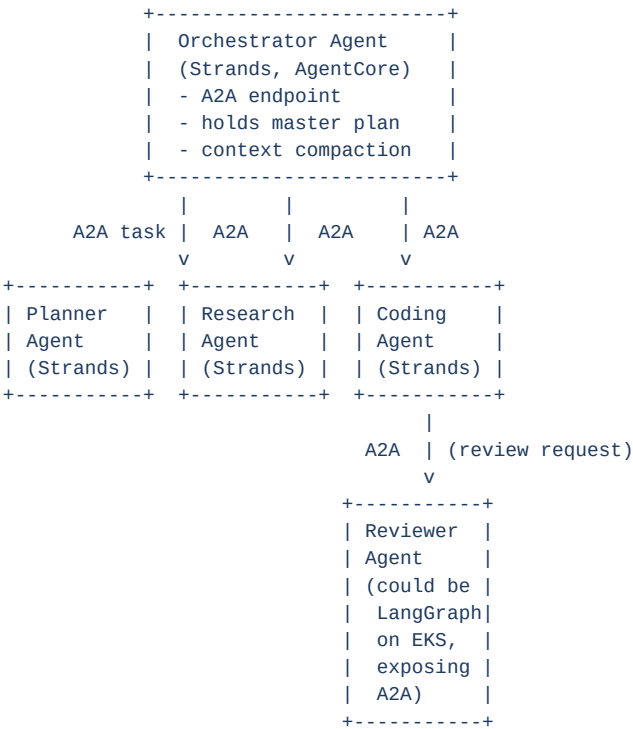
```
Namespace pattern: {tenant_id}/{scope}/{scope_id}

acme-corp/session/conv-8f3a...    <- working memory, this conversation
acme-corp/project/proj-finance-q3 <- shared project memory
acme-corp/user/user-jsmith        <- user-level semantic memory
acme-corp/org/global              <- organizational knowledge base
```

Retrieval for a given request queries namespaces in this order, with results merged and re-ranked by the relevance scoring model from prior research (Part 9.2): semantic similarity (40-60%), recency/validity (15-25%), importance (10-20%), source trust tier (5-10%), access frequency (5-10%).

3.3 Multi-Agent Topology with Strands + A2A

The manager/planner/research/coding/reviewer topology from prior research (Part 11) is implemented as a set of independently deployed Strands agents on AgentCore Runtime, coordinated via A2A. The topology below is illustrative for a software-engineering assistant; the pattern generalizes to other domains by substituting specialist agents.



- Each agent:
- Independently deployed on AgentCore Runtime (or elsewhere, if A2A-compliant)
 - Has its own AgentCore Memory namespace (scope: session, prefixed by tenant)
 - Has its own IAM role -> own MCP tool subset via AgentCore Gateway
 - Emits OTel spans correlated via shared trace_id (Section 2.9)
 - Returns compacted A2A results to caller (Section 2.6, context compaction)

Shared state vs. private state (per prior research Section 11.2)

State Type	Implementation	Notes
Blackboard (shared plan)	Aurora row, optimistic locking via version column	Orchestrator and planner read/write the master plan; version conflicts trigger re-read + retry
Per-agent private context	AgentCore Memory, session-scoped namespace	Not visible to other agents; cleared per prior research TTL policy at session end unless promoted to episodic memory
Handoff contracts (A2A task payloads)	JSON Schema, versioned, stored alongside A2A agent cards	Formal schema per prior research Section 11.2: task description, completed work, outstanding questions, required tools
Cross-agent event notifications	EventBridge (AsyncAPI-documented)	E.g., 'reviewer rejected coding agent's output'—triggers orchestrator re-planning without polling

3.4 Agent Recovery & Durable Execution

AgentCore Runtime's managed checkpointing handles step-level recovery (per prior research Section 4.2/4.3, this is 'Step-Level Checkpoints'—medium overhead, high recovery fidelity). The platform layers two additional mechanisms to approach the harder problem of fine-grained recovery (Open Problem B.3).

Mechanism 1: Idempotency keys on all state-changing tool calls

Every MCP tool definition for a state-changing operation requires an `idempotency_key` parameter (Section 2.5 example). The orchestrator generates this key once, before the first attempt, and includes it in retries. Lambda tool implementations check a DynamoDB idempotency table (key -> result, 24h TTL) before executing—directly preventing AP-13 (Non-Idempotent Retry).

Mechanism 2: Sub-step progress events for long-running tools

For tool calls expected to run longer than ~30 seconds (e.g., multi-file code generation, large report generation), the tool implementation emits progress events to EventBridge (AsyncAPI-documented: `ToolProgressEvent{tool_call_id, step, total_steps, partial_result_ref}`). On recovery, the orchestrator can query the last emitted progress event for an in-flight idempotency key and decide whether to resume from that point (if the tool supports resumption) or restart with the partial result as additional context.

DECISION: This does not fully solve Open Problem B.3—sub-step resumption is only possible for tools explicitly designed to support it, and most third-party MCP tools will not. The honest position: idempotency keys make retries *safe* (no duplicate side effects); progress events make restarts *cheaper* (less wasted work) where tools support it. Neither achieves true sub-step checkpointing for arbitrary tool operations—consistent with the assessment that this remains a genuinely open problem industry-wide.

PART 4

Security, Governance & Anti-Lock-In Strategy

This section closes the loop with prior research: every threat from the security threat model (prior research Part 12) and every anti-pattern (prior research Part D.5) is mapped to a specific control in this architecture.

4.1 Identity, Auth & Tenant Isolation

Identity Concern	Mechanism	Standard
End-user authentication	Amazon Cognito User Pools	OIDC / OAuth 2.0
Service-to-service auth (Layer 2 -> Layer 3)	IAM roles + SigV4, or OIDC token passthrough	OIDC (portable) or SigV4 (AWS-native, isolated to internal hop)
Agent identity (A2A)	Each agent has an A2A agent card with a verifiable identity (signed JWT or mTLS client cert)	A2A spec identity extensions
Tool authorization (MCP)	AgentCore Gateway maps OIDC scopes to MCP tool access via IAM policy	MCP + OAuth2 scopes
Tenant scoping	tenant_id claim in OIDC token, propagated as a required parameter/attribute through API Gateway -> Strands agent -> every storage query and memory namespace	Custom claim, OIDC-compatible

The one AWS-specific hop is service-to-service auth using IAM/SigV4 between internal components (e.g., Lambda-to-Lambda, AgentCore Runtime-to-Gateway). This is intentional: it is an internal implementation detail invisible to clients, agents, and tools, all of which interact via OIDC and A2A/MCP identity. If internal components were re-hosted off AWS, this hop would move to mTLS or another internal auth mechanism without affecting any external contract.

4.2 Governance & Compliance Mapping

Mapping the compliance matrix from prior research (Part 13.2) to concrete AWS mechanisms:

Regulation	Memory/Data Implication	AWS Mechanism
GDPR / CCPA	Right to erasure across all stores	Step Functions deletion-cascade workflow (Section 2.10) spanning Aurora, S3, OpenSearch, AgentCore Memory, Neptune; SLA-tracked via DynamoDB DeletionRequest table
Data residency	Region-pinned storage and processing	Per-tenant region configuration; AgentCore Runtime, Aurora, S3, and Bedrock model endpoints deployed per-region; tenant routing at API Gateway
SOC 2 / ISO 27001	Access controls, audit trails, encryption	IAM least-privilege + AWS Config rules for drift detection; CloudTrail + EventBridge audit pipeline (Section 2.10); KMS CMKs
HIPAA (if applicable)	PHI handling, BAA-covered services	Use only HIPAA-eligible AWS services (Bedrock, Aurora, S3, Lambda, AgentCore are HIPAA-eligible under BAA); Bedrock Guardrails for PHI redaction in agent outputs
EU AI Act (high-risk AI)	Explainability, human oversight for automated decisions	Full OTel trace export (Section 2.9) provides decision audit trail; AG-UI human-in-the-loop interrupts (Section 2.2) for decisions requiring approval

4.3 Anti-Pattern Cross-Reference

Direct mapping from the 15-pattern anti-pattern catalog (prior research Part D.5) to the specific architectural control in this design.

Code	Anti-Pattern	Architectural Control
AP-01	Context Window Dependence	Layered context assembly pipeline (Section 3.1); rolling summarization stored in Aurora, never relies on AgentCore session persistence alone
AP-02	Unlimited Memory Growth	Mandatory importance_score and ttl fields in memory schema (Section 3.2); periodic pruning Lambda driven by importance + TTL
AP-03	Stale Memory Retrieval	valid_from/valid_until fields + recency-weighted retrieval scoring (Section 3.2)
AP-04	Memory Poisoning via Tool Output	Provenance trust-tier system (Section 3.2); tool-derived (tier-3) memories require review before promotion to trusted status
AP-05	Excessive Summarization Loss	Anchor-fact preservation rule in summarization Lambda (per prior research D.2); numeric/date/decision content excluded from paraphrasing
AP-06	Trace Explosion	ADOT Collector tail-based sampling policy (Section 2.9): 100% for errors/high-latency, 1-5% for normal traces
AP-07	Missing Checkpointing	AgentCore Runtime managed checkpointing (Section 2.4) as the default for any multi-step agent
AP-08	Session-Server Coupling	AgentCore Runtime is inherently stateless-per-invocation with externalized session state; no agent code holds in-process state across invocations
AP-09	Artifact Duplication	S3 content-addressable storage (SHA-256 key) with artifact_versions table (per prior research Part 3.2); automatic dedup on identical content hash
AP-10	Project / Knowledge Fragmentation	Project namespace hierarchy (Section 3.2) with cross-project retrieval permitted by RBAC, avoiding artificial silos
AP-11	No Conflict Resolution on Memory Write	check_conflicts() in memory adapter (Section 2.7/3.2), invoked on every semantic-memory write
AP-12	Reasoning Trace as System Prompt Leak	AG-UI adapter (Section 2.2) filters trace events before client delivery: tool names and outcomes only, system prompts and raw parameters never forwarded
AP-13	Non-Idempotent Retry	Mandatory idempotency_key on all state-changing MCP tools (Section 2.5/3.4), enforced via DynamoDB idempotency table
AP-14	Deletion Without Cascade	Step Functions deletion-cascade workflow (Section 2.10/4.2)
AP-15	Cross-Tenant Query Without Scoping	tenant_id-prefixed namespaces (Section 3.2) + query-level scoping (Section 2.10) as defense-in-depth

4.4 Portability & Exit-Strategy Design

A practical exit strategy is not a document you write once—it's a set of artifacts that stay current because they are used in normal operations. This architecture treats the following as living artifacts:

- **OpenAPI/AsyncAPI specs are the source of truth** for Layer 2—API Gateway configuration is generated from them, not the reverse. The specs work regardless of which gateway implementation reads them.
- **MCP tool manifests (Layer 4)** are versioned independently of AgentCore Gateway configuration—the manifests are portable; the Gateway-specific deployment config is a thin adapter.
- **A2A agent cards (Layer 5)** describe agent capabilities in the standard format regardless of host—an agent registry query returns the same card whether the agent runs on AgentCore or elsewhere.
- **The MemoryProvider interface (Layer 6)** is tested against at least two implementations (AgentCore Memory + one alternative, e.g., a Postgres-based adapter used in local development/testing)—this is the practical guarantee that the abstraction is real, not aspirational.
- **OTel configuration (Layer 8)** is validated periodically by running the ADOT Collector with a secondary exporter to a non-AWS backend (even in a test environment)—proving the 'swap the exporter' claim rather than assuming it.
- **IaC modules are organized by layer**, not by AWS service—the Terraform/CDK module for 'Layer 4: Tools' contains AgentCore Gateway resources today, but its inputs/outputs are defined in terms of 'MCP tool endpoints', making a future module swap a contained change.

DECISION: The single highest-value exit-strategy practice: maintain a working (even if lower-performance) alternative implementation of the MemoryProvider interface and exercise it in CI. Prior research identified memory infrastructure as carrying the highest combination of lock-in risk and migration cost (Section 1.3)—a tested alternative is the difference between 'a multi-quarter migration project' and 'a configuration change plus a data migration job'.

Appendix: Service & Standards Reference

A.1 Complete AWS Service Inventory

Layer	AWS Service	Open Standard Interface	Primary Role
1. Client	API Gateway (WebSocket/HTTP streaming)	AG-UI	Event stream adapter to clients
2. API/Gateway	Amazon API Gateway (HTTP API)	OpenAPI 3.1	Synchronous REST contracts
2. API/Gateway	AWS AppSync Events / EventBridge	AsyncAPI 3.0	Event-driven contracts
2. API/Gateway	Amazon Cognito	OIDC / OAuth2	Identity provider
3. Agent Runtime	Amazon Bedrock AgentCore Runtime	A2A (via Strands)	Agent hosting, checkpointing, scaling
3. Agent Runtime	Strands Agents 1.0	A2A, OTel-native	Agent orchestration framework
3. Agent Runtime	Amazon Bedrock	(model API)	Foundation model access
3. Agent Runtime	Amazon Bedrock Guardrails	(policy config)	Content safety, PII redaction
4. Tools	Amazon Bedrock AgentCore Gateway	MCP	Tool exposure from Lambda/APIs
4. Tools	AWS Lambda	(implementation)	Tool execution
5. Multi-Agent	Amazon Bedrock AgentCore Runtime (per agent)	A2A	Specialist agent hosting
6. Memory	Amazon Bedrock AgentCore Memory	(provider-agnostic adapter)	Short & long-term memory
7. Data	Aurora PostgreSQL Serverless v2 (+pgvector)	SQL / pgvector	Conversations, metadata, vectors
7. Data	Amazon S3	S3 API / CloudEvents (audit)	Artifacts, documents, audit archive
7. Data	Amazon OpenSearch Service	OpenSearch API	Full-text & hybrid search
7. Data	Amazon Neptune (optional)	Gremlin / openCypher	Knowledge graph (Graph RAG)
7. Data	Amazon ElastiCache (Redis OSS)	Redis protocol	Session cache
8. Observability	AWS Distro for OpenTelemetry (ADOT)	OpenTelemetry / OTLP	Trace/metric/log collection
8. Observability	Amazon CloudWatch, AWS X-Ray	(default export target)	Default telemetry backend
9. Security	AWS IAM	(policy engine)	Authorization
9. Security	AWS KMS	(encryption)	Encryption key management
9. Security	AWS Step Functions	(orchestration)	Deletion cascade, long workflows
9. Security	AWS CloudTrail + EventBridge	CloudEvents	Audit logging

A.2 Standards Quick Reference

Standard	Governing Body / Origin	Maturity (as of mid-2026)	Role in This Architecture
MCP (Model Context Protocol)	Anthropic (open-sourced)	Production, broad ecosystem adoption	Tool/data access contract (Layer 4)

Standard	Governing Body / Origin	Maturity (as of mid-2026)	Role in This Architecture
A2A (Agent2Agent)	Google, contributed to Linux Foundation	Production; 150+ org adoption, AWS/Azure/GCP support	Agent coordination contract (Layer 5)
AG-UI	CopilotKit	Production, growing adoption	Agent-to-frontend event protocol (Layer 1)
OpenTelemetry	CNCF	Production, industry standard	Observability instrumentation (Layer 8)
OpenAPI 3.1	OpenAPI Initiative / Linux Foundation	Production, industry standard	Synchronous API contracts (Layer 2)
AsyncAPI 3.0	AsyncAPI Initiative / Linux Foundation	Production, growing adoption	Event-driven API contracts (Layer 2)
CloudEvents	CNCF	Production, industry standard	Audit/event format (Layer 7/9)
OIDC / OAuth 2.0	OpenID Foundation / IETF	Production, ubiquitous	Identity (Layer 2/9)

Appendix: Infrastructure-as-Code Skeleton

Illustrative module structure organized by architectural layer (per the portability principle in Section 4.4)—each module's interface is defined in standards terms, with AWS resources as the implementation.

```

infrastructure/
+-- modules/
|   +-- layer1-client/           # AG-UI adapter
|   |   +-- main.tf             # API Gateway WS + Lambda adapter
|   |   +-- variables.tf        # inputs: agentcore_runtime_endpoint
|   |   +-- outputs.tf         # outputs: ag_ui_websocket_url
|   |
|   +-- layer2-api-gateway/      # OpenAPI/AsyncAPI contracts
|   |   +-- main.tf             # API Gateway from openapi.yaml
|   |   +-- openapi.yaml        # <- source of truth
|   |   +-- asyncapi.yaml       # <- source of truth
|   |   +-- cognito.tf
|   |
|   +-- layer3-agent-runtime/    # AgentCore + Strands
|   |   +-- main.tf             # AgentCore Runtime resources
|   |   +-- agents/
|   |   |   +-- orchestrator/    # Strands agent container build
|   |   |   +-- planner/
|   |   |   +-- research/
|   |   |   +-- coding/
|   |   |   +-- reviewer/
|   |   +-- variables.tf        # inputs: mcp_gateway_endpoint,
|   |   |                               # memory_provider_config,
|   |   |                               # a2a_registry_endpoint
|   |
|   +-- layer4-tools-mcp/        # AgentCore Gateway (MCP)
|   |   +-- main.tf
|   |   +-- tool-manifests/      # <- portable MCP tool definitions
|   |   |   +-- create_support_ticket.json
|   |   |   +-- ...
|   |   +-- lambda-implementations/
|   |
|   +-- layer5-multiagent-a2a/    # A2A registry
|   |   +-- main.tf             # DynamoDB-backed agent registry
|   |   +-- agent-cards/        # <- portable A2A agent cards
|   |
|   +-- layer6-memory/           # Memory abstraction
|   |   +-- main.tf             # AgentCore Memory resources
|   |   +-- adapters/
|   |   |   +-- agentcore_adapter.py # default
|   |   |   +-- mem0_adapter.py      # swap-out, tested in CI
|   |   |   +-- postgres_adapter.py  # swap-out, used in local dev
|   |   +-- schema/
|   |   |   +-- memory_record.json   # <- portable schema (Section 3.2)
|   |
|   +-- layer7-data/             # Aurora, S3, OpenSearch, Neptune
|   |   +-- aurora.tf
|   |   +-- s3.tf
|   |   +-- opensearch.tf
|   |   +-- neptune.tf          # optional
|   |
|   +-- layer8-observability/     # ADOT Collector config
|   |   +-- main.tf
|   |   +-- adot-collector-config.yaml # <- exporter swap point
|   |
|   +-- layer9-security/         # IAM, KMS, Step Functions, audit
|   |   +-- iam.tf
|   |   +-- kms.tf
|   |   +-- deletion-cascade.tf   # Step Functions workflow
|   |   +-- audit-pipeline.tf
|
+-- environments/
|   +-- dev/

```

```
| +-- staging/  
| +-- prod/  
|  
+-- ci/  
    +-- memory-adapter-portability-test.yml # exercises 2+ adapters
```

PRINCIPLE: This module layout is the operational expression of Principle P1 (protocols at the boundary). Each module's `variables.tf/outputs.tf` define inputs and outputs in terms of standard interfaces (endpoints, manifests, schemas)—the `.tf` resource blocks inside are the only AWS-specific content, and they are the only thing that changes if a layer's implementation is later replaced.

Closing Statement

This architecture deliberately chooses AWS AgentCore and Strands as the path of least resistance for an AWS-native deployment—both are well-supported, actively developed (Strands 1.0 shipped May 2026 with multi-agent and A2A support), and genuinely reduce operational burden. The standards-first framing is not a rejection of that choice; it is the discipline that ensures the choice remains a choice. Every interface that matters—what a client sees, what a tool looks like, how agents talk to each other, how telemetry is structured, how memory is accessed—is defined by an open standard first, with AWS providing one (very good) implementation of each. Twelve months from now, if a component needs to change—for cost, capability, regulatory, or strategic reasons—the change should be localized to one layer, validated against an interface that was true from day one.